arXiv:2106.02852v2 [cs.CV] 4 Apr 2022

Patch Slimming for Efficient Vision Transformers

Yehui Tang[1,2], Kai Han[2], Yunhe Wang[2,*], Chang Xu[3],
Jianyuan Guo[2,3], Chao Xu[1], Dacheng Tao[4],
[1]School of Artificial Intelligence, Peking University. [2]Huawei Noah's Ark Lab.
[3]School of Computer Science, University of Sydney. [4]JD Explore Academy, China.
yhtang@pku.edu.cn, {kai.han, yunhe.wang}@huawei.com, dacheng.tao@gmail.com.

Abstract

This paper studies the efficiency problem for visual transformers by excavating redundant calculation in given networks. The recent transformer architecture has demonstrated its effectiveness for achieving excellent performance on a series of computer vision tasks. However, similar to that of convolutional neural networks, the huge computational cost of vision transformers is still a severe issue. Considering that the attention mechanism aggregates different patches layer-by-layer, we present a novel patch slimming approach that discards useless patches in a top-down paradigm. We first identify the effective patches in the last layer and then use them to guide the patch selection process of previous layers. For each layer, the impact of a patch on the final output feature is approximated and patches with less impact will be removed. Experimental results on benchmark datasets demonstrate that the proposed method can significantly reduce the computational costs of vision transformers without affecting their performances. For example, over 45% FLOPs of the ViT-Ti model can be reduced with only 0.2% top-1 accuracy drop on the ImageNet dataset.

1. Introduction

Recently, transformer models have been introduced into the field of computer vision and achieved high performance in many tasks such as object recognition [6], image process [2], and video analysis [21]. Compared with the convolutional neural networks (CNNs), the transformer architecture introduces less inductive biases and hence has larger potential to absorb more training data and generalize well on more diverse tasks [6, 11, 27, 35, 38, 43]. However, similar to CNNs, vision transformers also suffer high computational cost, which blocks their deployment on resource-limited devices such as mobile phones and various IoT devices. To apply a deep neural network in such real scenarios, massive model compression algorithms have been proposed to reduce the required computational cost [24, 42]. For example, quantization algorithms approximate weights and intermediate features maps in neural networks with low-bit data [3, 32]. Knowledge distillation improves the performance of a compact network by transferring knowledge from giant models [17, 22].

In addition, network pruning is widely explored and used to reduce the neural architecture by directly removing useless components in the pre-defined network [13, 14, 25, 28]. Structured pruning discards whole contiguous components of a pre-trained model, which has attracted much attention in recent years, as it can realize acceleration without specific hardware design. In CNNs, removing a whole filter for improving the network efficiency is a representative paradigm, named channel pruning (or filter pruning) [16, 25]. For example, Liu *et al*. [25] introduce scaling factors to control the information flow in the neural network and filters with small factors will be removed. Although the aforementioned network compression methods have made tremendous efforts for deploying compact convolutional neural networks, there are only few works discussing how to accelerate vision transformers.

Different from the paradigm in conventional CNNs, the vision transformer splits the input image into multiple patches and calculates the features of all these patches in parallel. The attention mechanism will further aggregate all patch embeddings into visual features as the output. Elements in the attention map reflect the relationship or similarity between any two patches, and the largest attention value for constructing the feature of an arbitrary patch is usually calculated from itself. Thus, we have to preserve this information flow in the pruned vision transformers for retaining the model performance, which cannot be guaranteed in the conventional CNN channel pruning methods. Moreover, not all the manually divided patches are informative enough and deserve to be preserved in all layers, *e.g*., some patches are redundant with others. Hence we consider developing a patch slimming approach that can effectively identify and

*Corresponding author.

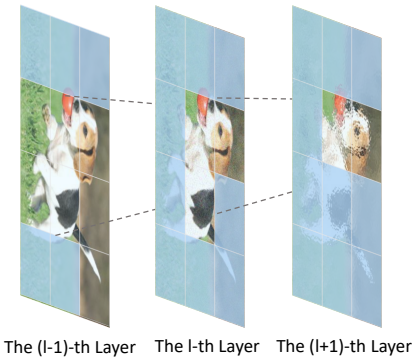


Figure 1. The diagram of patch slimming for vision transformers.

remove redundant patches.

In this paper, we present a novel patch slimming algorithm for accelerating the vision transformers. In contrast to existing works focusing on the redundancy in the network channel dimension, we aim to explore the computational redundancy in the patches of a vision transformer (as shown in Figure 1. The proposed method removes redundant patches from the given transformer architecture in a top-down framework, in order to ensure the retained high-level features of discriminative patches can be well calculated. Specifically, the patch pruning will execute from the last layer to the first layer, wherein the useless patches are identified by calculating their importance scores to the final classification feature (*i.e.*, class token). To guarantee the information flow, a patch will be preserved if the patches in the same spatial location are retained by deeper layers. For other patches, the importance scores determine whether they are preserved, and patches with lower scores will be discarded. The whole pruning scheme for vision transformers is conducted under a careful control of the network error, so that the pruned transformer network can maintain the original performance with significantly lower computational cost. Extensive experiments validate the effectiveness of the proposed method for deploying efficient vision transformers. For example, our method can reduce more than 45% FLOPs of the ViT-Ti model with only 0.2% top-1 accuracy loss on the ImageNet dataset.

2. Related work

Structure pruning for CNNs. Channel pruning discards the entire convolution kernels [23] to accelerate the inference process and reduce the required memory cost [15, 23, 25, 33, 34]. To identify the redundant filters, massive methods have been proposed. Wen *et al*. [41] add a group-sparse regularization on the filters and remove filters with small norm. Beyond imposing sparsity regularization on the filters directly, Liu *et al*. [25] introduce extra scaling factors to each channel and these scaling factors are trained to be sparse. Filters with small scaling factors has less impact on the network output and will be removed for accelerating inference. He *et al*. [15] rethink the criterion that filters with small norm values are less important and propose to discard the filters having larger similarity to others. To maximally excavate redundancy, Tang [37] set up a scientific control to alleviate the distribution of irrelevant factors and remove filters with little relation to the given task. In the conventional channel pruning for CNNs, channels in different layers have no one-to-one relationship, and then the choice of effective channels in a layer has little impact on that in other channels.

Structure pruning for transformers. In the transformer model for NLP tasks, a series of works focus on reducing the heads in the multi-head attention (MSA) module. For example, Michel *et al*. [29] observes that removing a large percentages of heads in the pre-trained BERT [5] models has limited impact on its performance. Voita *et al*. [39] analyze the role of each head in the transformer and evaluate their contribution to the model performance. Those heads with less contributions will be reduced. Besides the MSA module, the neurons in the multilayer perceptron (MLP) module are also pruned in [1]. Designed for vision transformers, VTP [44] reduces the number of embedding dimensions by introducing control coefficients and removes neurons with small coefficients. Different from them, the proposed patch slimming explores the redundancy from a new perspective by considering the information integration of different patches in a vision transformer. Actually, reducing patches can be also combined with pruning in other dimensions to realize higher acceleration.

3. Patch Slimming for Vision Transformer

In this section, we introduce the scheme of pruning patches in vision transformers. We first review the vision transformer briefly and then introduce the formulation of patch slimming.

In vision transformer, the input image is split into N patches and then fed into transformer model for representation learning. For an L-layer vision transformer model, the multihead self-attention (MSA) modules and multi-layer perceptron (MLP) modules are its main components occupying most of the computational cost. Denoting $Z_{l-1}, Z'_l \in \mathbb{R}^{N\times d}$ as the input and the intermediate features of the l-th layer, the MSA and MLP modules can be formulated as:

$$\begin{aligned}&\mathrm{MSA}(Z_{l-1})\\&\quad=\mathrm{Concat}\left[\mathrm{softmax}\left(\frac{Q_l^h {K_l^h}^\top}{\sqrt{d}}\right)V_l^h\right]_{h=1}^{H} W_l^o,\\&\mathrm{MLP}(Z'_l)=\phi(Z'_l W_l^a)W_l^b,\end{aligned} \tag{1}$$

where d is embedding dimension, H is the number of heads, $Q_l^h = Z_{l-1}W_l^{hq}$, $K_l^h = Z_{l-1}W_l^{hk}$, and $V_l^h = Z_{l-1}W_l^{hv}$

are the query, key and value of the h-th head in the l-th layer, respectively. W_l^a, W_l^b are the weights for linear transformation and $\phi(\cdot)$ is the non-linear activation function (*e.g.*, GeLU). Most of recent vision transformer models are constructed by stacking MSA and MLP modules alternately and a block $\mathcal{B}_l(\cdot)$ is defined as $\mathcal{B}_l(Z_{l-1}) = \text{MLP}(\text{MSA}(Z_{l-1}) + Z_{l-1}) + Z'_l$.

As discussed above, there is considerable redundant information existing in the patch level of vision transformers. To further verify this phenomenon, we calculate the average cosine similarity between patches within a layer, and show how similarity vary w.r.t. layers in Figure 3. The similarity between patches increase rapidly as layers increase, and the average similarity even exceed 0.8 in deeper layers. The high similarity implies that patches are redundant especially in the deeper layers and removing them will not obviously affect the feature calculation.

Patch slimming aims to recognize and discard redundant patches for accelerating the inference process (as shown in Figure 1). Here we use a binary vector $\boldsymbol{m}_l \in \{0,1\}^N$ to indicate whether a patch is preserved or not, the pruned MSA and MLP modules can be formulated as follows:

$$\begin{aligned}
&\widehat{\text{MSA}}_l(\widehat{Z}_{l-1}) \\
&= \text{Concat}\left[\text{diag}(\boldsymbol{m}_l)\text{softmax}\left(\frac{Q_l^h {K_l^h}^\top}{\sqrt{d}}\right)V_l^h\right]_{h=1}^{H} W_l^o, \\
&\widehat{\text{MLP}}_l(\widehat{Z}'_l) = \text{diag}(\boldsymbol{m}_l)\phi(\widehat{Z}'_l W_l^a)W_l^b,
\end{aligned} \tag{2}$$

where $\text{diag}(\boldsymbol{m}_l)$ is a diagonal matrix whose diagonal line is composed of elements in $\boldsymbol{m}_l$. Specifically, $\boldsymbol{m}_{l,i} = 0$ indicates that the i-th patch in the l-th layer is pruned. $\widehat{Z}_{l-1}$, $\widehat{Z}'_l$ are the input and the intermediate features of the l-th layer in a pruned vision transformer. Then the pruned block is defined as $\widehat{\mathcal{B}}_l(\widehat{Z}_{l-1}) = \widehat{\text{MLP}}_l(\widehat{\text{MSA}}_l(\widehat{Z}_{l-1}) + \widehat{Z}_{l-1}) + \widehat{Z}'_l$.

In practical implementation, only the effective patches of input feature $\widehat{Z}_{l-1}$ are selected to calculate queries, and then all the subsequent operations are only implemented on these effective patches. Thus, the computation of the pruned patches can be avoided [1].

Computation Efficiency. Compared with the original block $\mathcal{B}_l(\cdot)$, the pruned $\widehat{\mathcal{B}}_l(\cdot)$ can save a large amount of computational cost. Given a block $\mathcal{B}(\cdot)$ with N patches and d-dimension embedding, the computational costs of MLP (2-layers with hidden dimension d') and MSA are $(2Ndd')$ and $(2N^2d + 4Nd^2)$, respectively. After pruning $\eta\%$ patches, all the computational components in MLP are pruned, and then $\eta\%$ FLOPs in the MLP module are reduced. For the MSA module, the cost of calculating query, attention map and output projection can be reduced, and then $\eta\%(2N^2d + 2Nd^2)$ FLOPs is reduced.

[1] According to $\boldsymbol{m}_l$, only effective patches from the shortcut branch are added to the output of pruned MSA, while the output of pruned MLP is padded with zeros before added to the shortcut.

4. Excavating Redundancy via Inverse Pruning

In this section, we present the top-down framework to prune patches in the vision transformer, and provide an effective importance score estimation of each patch.

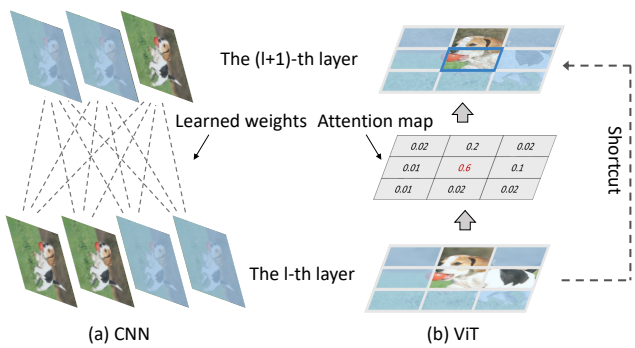


Figure 2. The comparison between channel pruning in CNNs and patch pruning in vision transformers.

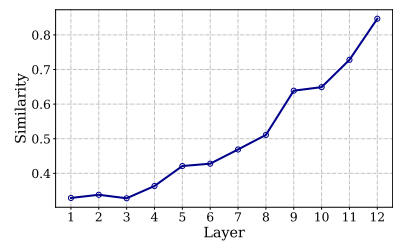


Figure 3. The average similarity of different patches varies w.r.t. network depth in the ViT-Base model.

4.1. Top-Down Pruning

For patch slimming in vision transformer, we adopt a top-down manner to prune patches layer-by-layer. It is a natural choice with two reasons as described in the following.

For a CNN model, pruning channels in different layers independently can achieves high performance [25, 37]. However, this paradigm cannot work well in vision transformers. The main reason is that patches in different layers of a vision transformer are one-to-one corresponding. Figure 2 compares pruning channels in CNNs and pruning patches in vision transformers. As own in Figure 2(a), channels in adjacent layers of a CNN model are fully connected by learnable weights, and each channel contains information from the entire image. However, in the vision transformer (Figure 2(b)), different patches communicate with others by an attention map, which reflects the similarity between different patches. If patch i and patch j are more

similar, the corresponding value A_{lh}^{ij} tends to have a larger value. The diagonal elements A_{lh}^{ii} usually plays a dominant role, that is, a patch pays highest attention to the input at the position of itself. Besides, the shortcut connection directly copies the feature in the l-layer to the corresponding patches in the next layer. This one-to-one correspondence inspires us to preserve some important patches in the same spatial locations of different layers, which can guarantee the information propagation across layers.

Another characteristic of vision transformer is that deeper layers tend to have more redundant patches. The attention mechanism in the MSA module aggregates different patches layer-by-layer, and a large number of similar patches are produced in the process (as shown in Figure 3). It implies that more redundant patches can be safely removed in deeper layers, and fewer in shallower layers.

Based on the above analysis, we start the pruning procedure from the output layer, and then prune previous layers by transmitting the selected effective patches from top to down. Specially, all the patches preserved in the $(l+1)$-th layer will be also preserved in the l-th layer. Thus, this top-down pruning procedure can guarantee that shallow layers maintain more patches than the deep layers, which is consistent with the redundancy characteristic of vision transformer.

4.2. Impact Estimation

With the patch pruning scheme described in the above section, all that's left is to recognize redundant patches in a vision transformer, *i.e.*, find the optimal mask $\boldsymbol{m}_l$ in each layer. Our goal is to prune patches as many as possible to realize maximal acceleration, while maintaining the representation ability of the output feature. Actually, only a part of patch embeddings in the last layer are used to predict the labels of input images for a specific task. For example, in the image classification task, only a patch related to classification (*i.e.*, class token) is sent to the classifier for predicting labels. Other patches in the output layer can be removed safely without affecting network output. Supposing the first patch is the class token, we can get the mask in the last layer, *i.e.*, $\boldsymbol{m}_{L,1}=1$, and $\boldsymbol{m}_{L,i}=0, \forall\, i=2,3,\cdots,N$. Then for the other layers, the optimization object is formulated as follows:

$$\begin{aligned} &\min_{\boldsymbol{m}_1,\boldsymbol{m}_2,\cdots,\boldsymbol{m}_{L-1}} \sum_{l=1}^{L-1} \|\boldsymbol{m}_l\|_0, \\ &\text{s.t. } \boldsymbol{m}_l \in \{0,1\}^N, \\ &\mathcal{E}_L = \left\|\text{diag}(\boldsymbol{m}_L)\left(\widehat{Z}_L - Z_L\right)\right\|_F^2 \le \epsilon, \end{aligned} \tag{3}$$

where $\|\cdot\|_0$ is the ℓ_0-norm of a vector, *i.e.*, the number of non-zero elements. $\|\cdot\|_F$ is the Frobenius norm of a matrix. ϵ is the tolerable error. $\widehat{Z}_L$ and Z_L are output features of the pruned and unpruned transformers. Eq. 3 is hard to optimize directly, as it involve ℓ_0 optimization under constraint, which is non-convex, NP hard and requires combinatorial search [25]. To solve Eq. 3, we firstly define a signification score by approximating the impact of a patch on the reconstruction error $\mathcal{E}_L$ and then develop a pruning procedure.

The attention mechanism aggregates information from different patches to one patch, which is the main cause to produce redundant patch features. To focus on the attention layer for excavating redundant patches, we reformulate the definition of a block $\mathcal{B}(\cdot)$ in a simple formulation. Denoting $P_l^h = \text{softmax}\left(Q_l^h {K_l^h}^\top/\sqrt{d}\right)$, the MSA module in Eq. 1 can be formulated as:

$$\begin{aligned} \text{MSA}(Z_l) &= \text{Concat}\left[P_l^h V_l^h\right]_{h=1}^{H} W_l^o \\ &= \sum_{h=1}^{H} P_l^h V_l^h W_l^{ho} = \sum_{h=1}^{H} P_l^h Z_{l-1} W_l^{hv} W_l^{ho}, \end{aligned} \tag{4}$$

where $W_l^o = [W_l^{1o}; W_l^{2o}; \cdots; W_l^{Ho}]$, and $W_l^{ho} \in \mathbb{R}^{\frac{d}{H}\times d}$. Then the original block $\mathcal{B}_l(Z_{l-1})$ and pruned block $\widehat{\mathcal{B}}_l(Z_{l-1})$ can be represented as:

$$\begin{aligned} \mathcal{B}_l(Z_{l-1}) &= \mathcal{O}(\sum_{h=1}^{H} P_l^h Z_{l-1}, \{W_l\}), \\ \widehat{\mathcal{B}}_l(Z_{l-1}, \boldsymbol{m}_l) &= \mathcal{O}(\sum_{h=1}^{H} \text{diag}(\boldsymbol{m}_l) P_l^h Z_{l-1}, \{W_l\}) \end{aligned} \tag{5}$$

where $\mathcal{O}(\cdot, W_l)$ is composed of multiple linear projection matrices $\{W_l\}$ in the MSA and MLP module, as well as non-linear activation functions (*e.g.*, GeLU).

Based on the simplified formulation of a block (Eq. 5), we here explore how a patch in the t-th layer affects the error $\mathcal{E}_L$ (Eq 3) of effective patches in the last layer. We reverse the transformer and prune it from the last to the first layer sequentially. Thus when it comes to the t-th layer, all the deeper layers have been pruned. To approximate the significance of each token, we have the following theorem.

Theorem 1. *The impact of the t-th layer's patch on the final error $\mathcal{E}_L$ can be reflected by a significance metric $\boldsymbol{s}_t \in \mathbb{R}^N$. For the i-th patch in the t-th layer, we have*

$$\boldsymbol{s}_{t,i} = \sum_{h\in[H]^{L\sim t+1}} \left\|A_t^h[:,i] U_t^h[i,:]\right\|_F^2, \tag{6}$$

where $A_t^h = \prod_{l=t+1}^{L} diag(\boldsymbol{m}_l) P_l^h$ and $U_t^h = P_t^h |Z_{t-1}|$. $A_t^h[:,i]$ denotes the i-th column of A_t^h and $U_t^h[i,:]$ is i-th row of U_t^h. $[H]^{L\sim t+1}$ denotes all the attention heads in the $(t+1)$-th to L-th layer.

Proof. We use $\widehat{\mathcal{F}}_{l\sim t}(Z_{t-1}, \{\boldsymbol{m}_t\}_t^L)$ $(l > t)$ to denote feature of the l-th layer in a vision transformer, whose layers behind t-th layer have been pruned, while the previous layers has not pruned yet, *i.e.*, $\widehat{\mathcal{F}}_{L\sim t}(Z_{t-1}, \{\boldsymbol{m}_t\}_t^L) =$

$\hat{\mathcal{B}}_L \circ \hat{\mathcal{B}}_{L-1} \circ \cdots \circ \hat{\mathcal{B}}_t(Z_{t-1})$. When pruning the patch in the t-th layer, we compare effective patches of the last layer from two transformers to decide whether the t-th layer has been pruned. Then the error $\mathcal{E}_L$ is calculated as:

$$\begin{aligned}\mathcal{E}_L = ||\text{diag}(\boldsymbol{m}_L)[\widehat{\mathcal{F}}_{L\sim(t+1)}(\widehat{\mathcal{B}}_t(Z_{t-1})) \\ - \widehat{\mathcal{F}}_{L\sim(t+1)}(\mathcal{B}_t(Z_{t-1}))]||_F^2.\end{aligned} \tag{7}$$

The error $\mathcal{E}_L$ in the last layer can be represented by the patches in the $(L-1)$-th layer, *i.e.*,

$$\begin{aligned}\mathcal{E}_L &= ||\text{diag}(\boldsymbol{m}_L)P_L^h[\mathcal{O}(\sum_{h=1}^{H}\widehat{\mathcal{F}}_{(L-1)\sim(t+1)}(\widehat{\mathcal{B}}_t(Z_{t-1})))\\ &- \mathcal{O}(\sum_{h=1}^{H}\widehat{\mathcal{F}}_{(L-1)\sim(t+1)}(\mathcal{B}_t(Z_{t-1})))]||_F^2\\ &\leq C_L||\sum_{h=1}^{H}\text{diag}(\boldsymbol{m}_L)P_L^h|\widehat{\mathcal{F}}_{(L-1)\sim(t+1)}(\widehat{\mathcal{B}}_t(Z_{t-1}))\\ &- \widehat{\mathcal{F}}_{(L-1)\sim(t+1)}(\mathcal{B}_t(Z_{t-1}))|||_F^2,\end{aligned} \tag{8}$$

where $|\cdot|$ is the element-wisely absolute value. The inequality above comes the Lipschitz continuity [7, 8] of function $\mathcal{O}(\cdot)$ and C_L is the Lipschitz constant. Recalling that $\mathcal{O}(\cdot)$ is compose of multiple linear projections and non-linear activation function, the condition of Lipschitz continuity is satisfied [8]. E_L can be further transmitted to previous layers, and for the t-th layer we have

$$\mathcal{E}_L \leq \prod_{l=t+1}^{L} C_l|| \sum_{h\in[H]^{L\sim t}} \prod_{l=t+1}^{L}\text{diag}(\boldsymbol{m}_l)P_l^h \tag{9}$$

$$|\widehat{\mathcal{B}}_t(Z_{t-1}) - \mathcal{B}_t(Z_{t-1})|||_F^2 \tag{10}$$

$$\leq \prod_{l=t}^{L} C_l|| \sum_{h\in[H]^{L\sim t+1}} \prod_{l=t+1}^{L}\text{diag}(\boldsymbol{m}_l) \tag{11}$$

$$P_l^h\left(I_N - \text{diag}(\boldsymbol{m}_l)\right)P_t^h\left|Z_{t-1}\right|||_F^2 \tag{12}$$

$$= C_t'|| \sum_{h\in[H]^{L\sim t+1}} A_t^h(I_N - \text{diag}(\boldsymbol{m}_t))U_t^h||_F^2, \tag{13}$$

where $A_t^h = \prod_{l=t+1}^{L}\text{diag}(\boldsymbol{m}_l)P_l^h \in \mathbb{R}^{N\times N}$, $U_t^h = P_t^h\left|Z_{t-1}\right| \in \mathbb{R}^{N\times d}$, and $C_t' = \prod_{l=t}^{L} C_l$. $[H]^{l\sim t}$ denotes all the attention heads in the t-th to l-th layer. To investigate how each patch in the t-th layer affect the final error E_L, we expand Eq. 13 w.r.t. each element in the indicator $\boldsymbol{m}_l$. Denoting $\boldsymbol{m}_{l,i}$ as the i-th element in $\boldsymbol{m}_l$, $A_t^h[:,i]$ is the i-th column of A_t^h and $U_t^t[i,:]$ is i-th row of U_t^h, Eq. 13 can be written as:

$$\mathcal{E}_L \leq C_t'|| \sum_{h\in[H]^{L\sim t+1}} \sum_{i=1}^{N} A_t^h[:,i](1-\boldsymbol{m}_{t,i})U_t^h[i,:]||_F^2 \tag{14}$$

$$\leq C_t'\sum_{i=1}^{N}(1-\boldsymbol{m}_{t,i}) \sum_{h\in[H]^{L\sim t+1}} \left\|A_t^h[:,i]U_t^h[i,:]\right\|_F^2. \tag{15}$$

Then we get the importance of each patch, *i.e.*, $\boldsymbol{s}_{t,i} = \sum_{h\in[H]^{L\sim t+1}} \left\|A_t^h[:,i]U_t^h[i,:]\right\|_F^2$. □

For the i-th patch in the t-th layer, $\boldsymbol{s}_{t,i}$ reflects its impact on the effective output of the final layer. A larger $\boldsymbol{s}_{t,i}$ implies the corresponding patch has larger impact to the final error, which can reflect the importance of a patch to the model performance. The calculation of $\boldsymbol{s}_{t,i}$ involves all the attention maps in behind layers and the input feature of the current layer. Before pruning the current layer, we randomly sample a subset of training dataset to calculate the significance scores $\boldsymbol{s}_t$ and the average $\boldsymbol{s}_t$ over these data is adopted. The obtained $\boldsymbol{s}_t$ can be viewed as the real-number score for binary $\boldsymbol{m}_t$.

Algorithm 1 Patch Slimming for Vision Transformers.

Input: Training dataset $\mathcal{D}$, vision transformer $\mathcal{T}$ with L layers, patch masks $\{\boldsymbol{m}_l\}_{l=1}^{L}$, tolerant value ϵ, preserved patch's number r and search granularity r'.

Initialize $\boldsymbol{m}_{L,0}$ as 1 and other elements as 0.
for $l = L-1, \cdots, 1$ **do**
  Randomly sample a subset of training data to get the significance score $\boldsymbol{s}_l$ in the l-th layer;
  Set $\boldsymbol{m}_l = \boldsymbol{m}_{l+1}$, $\mathcal{E}_l = +\infty$, $r = 0$;
  while $\mathcal{E}_{l+1} > \epsilon$ **do**
    Set r elements in $\boldsymbol{m}_{l,i}$ to 1 according to positions of the largest r scores $\boldsymbol{s}_{l,i}$.
    Fine-tune l-th layer $\mathcal{B}_l(Z_{l-1})$ for a few epochs.
    Calculate error $\mathcal{E}_{l+1}$ in the $(l+1)$-th layer.
    $r = r + r'$.
  end while
end for

Output: The pruned vision transformer.

4.3. Pruning Procedure

Here we conclude the overall pipeline of the proposed patch slimming method.

We start from the output layer and prune the previous layers layer-by-layer from top to down. Specially, all the patches preserved in the $(l+1)$-th layer will be also preserved in the l-th layer. The other patches are greedily selected according to their impact scores $\boldsymbol{s}_{l,i}$, wherein patches

Table 1. Comparison of the pruned vision transformers with different methods on ImageNet. 'FLOPs ↓' denotes the reduction ratio of FLOPs.

Model	Method	Top-1 Acc. (%)	Top-5 Acc. (%)	FLOPs (G)	FLOPs ↓ (%)	Throughput (image / s)	Throughput ↑ (%)
ViT (DeiT)-Ti	Baseline	72.2	91.1	1.3	0	2536	0
	SCOP [37]	68.9 (-3.3)	89.0 (-2.1)	0.8	38.4	3372	33.0
	PoWER [10]	69.4 (-2.8)	89.2 (-1.9)	0.8	38.4	3304	30.3
	HVT [30]	69.7 (-2.5)	89.4 (-1.7)	0.7	46.2	3524	38.9
	PS-ViT (Ours)	**72.0** (-0.2)	**91.0** (-0.1)	0.7	46.2	3576	41.0
	DPS-ViT (Ours)	**72.1** (-0.1)	**91.1** (-0.0)	0.6	53.8	3639	43.5
ViT (DeiT)-S	Baseline	79.8	95.0	4.6	0	940	0
	SCOP [37]	77.5 (-2.3)	93.5 (-1.5)	2.6	43.6	1310	39.4
	PoWER [10]	78.3 (-1.5)	94.0 (-1.0)	2.7	41.3	1295	37.8
	HVT [30]	78.0 (-1.8)	93.8 (-1.2)	2.4	47.8	1335	42.1
	PS-ViT (Ours)	**79.4** (-0.4)	**94.7** (-0.3)	2.6	43.6	1321	40.5
	DPS-ViT (Ours)	**79.5** (-0.3)	**94.8** (-0.2)	2.4	47.8	1342	42.8
ViT (DeiT)-B	Baseline	81.8	95.6	17.6	0	292	0
	SCOP [37]	79.7 (-2.1)	94.5 (-1.1)	10.2	42.0	403	38.1
	PoWER [10]	80.1 (-1.7)	94.6 (-1.0)	10.4	39.2	397	35.8
	VTP [44]	80.7 (-1.1)	95.0 (-0.6)	10.0	43.2	412	41.0
	PS-ViT (Ours)	**81.5** (-0.3)	**95.4** (-0.2)	9.8	44.3	414	41.8
	DPS-ViT (Ours)	**81.6** (-0.2)	**95.4** (-0.2)	9.4	46.6	413	41.3
T2T-ViT-14	Baseline	81.5	95.4	5.2	0	764	0
	PoWER [10]	79.9 (-1.6)	94.4 (-1.0)	3.5	32.7	991	29.7
	PS-T2T (Ours)	**81.1** (-0.4)	**95.2** (-0.2)	3.1	40.4	1055	38.1
	DPS-T2T (Ours)	**81.3** (-0.2)	**95.3** (-0.1)	3.1	45.4	1078	41.1

with larger scores are preserved preferentially. Considering the reconstruction error $\mathcal{E}_{l+1}$ in the $(l+1)$-th layer is directly affected by the patch selection in the l-th layer, we use it to determine whether the l-th layer has already enough patches. In practice, we iteratively select r' important patches in each step and continue the selection process in the current layer until $\mathcal{E}_{l+1}$ is less than the given tolerate value ϵ. To make $\mathcal{E}_{l+1}$ well maintain the representation ability of current preserved patches, we fine-tune the current block $\widehat{\mathcal{B}}_l$ for a few epochs after each step of patch selection. Taking the original feature Z_{l-1} in the $(l-1)$-th layer as input, and the reconstruction error $\mathcal{E}_{l+1}$ as the objective, the parameters in the current block $\widehat{\mathcal{B}}_l$ are optimized. Note that the block $\widehat{\mathcal{B}}_l$ is a very small model with only one MSA and one MLP modules, the fine-tune process is very fast. After pruning, the mask $\boldsymbol{m}_l$ is fixed, and weight parameters in the vision transformer is further fine-tuned to be compatible with the efficient architecture. The procedure of patching slimming for vision transformer is summarized in Algorithm 1.

4.4. A Dynamic Variant

In the above procedure, whether a patch will be preserved is determined by the statistics over the training dataset. It exploits the commonalities of redundant filters from different input adequately. Besides, dynamic pruning is the improved version of static pruning methods, which selects different patches for each input image. The dynamic strategy has been widely explored for reducing channels of CNN models [9, 18, 36]. Similarly, the proposed patch slimming paradigm can be easily extended to the dynamic variant (dubbed as DPS-ViT), and here we present a simple implementation. Following [9], we insert a small module $\mathcal{G}$ in each block to predict which patch is effective. The module $\mathcal{G}$ composes of a downsampling layer, linear layer and activation functions, which takes the input feature $Z_{l-1} \in \mathbb{R}^{N \times d}$ as input and outputs the approximate significance score $\hat{\boldsymbol{s}}_l \in \mathbb{R}^N$. Recalling that the score of a patch $\boldsymbol{s}_l$ (Eq. 6) actually depends on the input images, the module $\mathcal{G}$ is trained to fit $\boldsymbol{s}_l$ calculated for each input instance in the training phase. At inference, only patches with large score $\hat{\boldsymbol{s}}_{l,i}$ are required to calculate. The dynamic strategy finds redundant patches of vision transformers depending on input data, which can excavate patch redundancy more adequately.

5. Experiments

In this section, we empirically investigate the effectiveness of the proposed patch slimming methods for efficient vision transformers (PS-ViT). We evaluate our method on the benchmark ImageNet (ILSVRC2012) [4] dataset, which contains 1000-class natural images, including 1.2M training images and 5k validation images. The proposed method is compared with SOTA pruning methods and we also conduct extensive ablation studies to better understand our method.

Table 2. Comparisons with SOTA transformer models on ImageNet.

Model	FLOPs (G)	Top-1 Acc. (%)
DeiT-S [38]	4.6	79.8
DeiT-B [38]	17.5	81.8
PVT-Small [40]	3.8	79.8
PVT-Medium [40]	6.7	81.2
PVT-Large [40]	9.8	81.7
T2T-ViT-14 [43]	5.2	81.5
T2T-ViT-19 [43]	8.9	81.9
T2T-ViT-24 [43]	14.1	82.3
TNT-S [12]	5.2	81.5
TNT-B [12]	14.1	82.9
Swin-T [26]	4.5	81.3
Swin-S [26]	8.7	83.0
Swin-B [26]	15.4	83.5
LV-ViT-S [20]	6.6	83.3
LV-ViT-M [20]	16.0	84.1
PS-LV-ViT-S (Ours)	4.7	82.4
DPS-LV-ViT-S (Ours)	4.5	82.9
PS-LV-ViT-M (Ours)	8.6	83.5
DPS-LV-ViT-M (Ours)	8.3	83.7

5.1. Experiments on ImageNet

We conduct experiments on the standard ViT models [6] (DeiT [38]), an improved variant network T2T-ViT [43] and the state-of-the art LV-ViT [20].

Implementation details. For a fair comparison, we follow the training and testing settings in the original papers [38, 43, 43], and the patch slimming is implemented based on the official pre-trained models. The global tolerant error is select from $\{0.01, 0.02\}$ to get models with different acceleration rates, and the search granularity r is set to 10. We fine-tune the current block for 3 epochs after each iteration of patch selection. After determining the proper patches in each layer, the pruned transformers are fine-tuned following the training strategy in [38]. All the experiments are conducted with PyTorch [31] and MindSpore [19] on NVIDIA V100 GPUs.

Competing methods. We compare our patch slimming with several representative model pruning methods including CNN channel pruning methods [37] and BERT pruning methods [10]. SCOP [37] is a SOTA network pruning method for reducing the channels of CNNs, and we re-implement it to reduce the patches in vision transformers. PoWER [10] accelerates BERT inference by progressively eliminating word-vector. HVT [30] directly designs efficient vision transformer architectures by progressively reducing the spatial dimensions through pooling operations.

Experimental results. The experimental results are shown in Table 1, where 'PS-' and 'DPS-' denote the proposed patch pruning method and its dynamic variant, respectively. We evaluate on three versions of DeiT [38] with different model sizes, *i.e.*, DeiT-Ti, DeiT-S, and DeiT-B.

Table 3. Learned patch pruning *vs*. uniform pruning.

Method	Top-1 Acc. (%)	Top-5 Acc. (%)	FLOPs (G)
Baseline	79.8	95.0	4.6
Uniform pruning	77.2	93.8	2.6
Ours	79.4	94.7	2.6

Our method achieve obviously higher performance compared to the existing methods. The SCOP method [37] designed for CNNs achieve poor performance when applied for reducing patches in a vision transformer, implying simply migrating the channel pruning methods cannot work well. PoWER [10] has a larger accuracy drop than our method, indicating the model compression method for NLP models is not optimal for CV models. Compared to the vision transformer structure pruning method VTP [44], our method investigates a new prospective by pruning patches and achieve higher accuracy with similar FLOPs.

As for T2T-ViT model, our method can reduce the FLOPs by 40.4% and only have a small accuracy decrease (0.4%), which is much better than the compared PoWER method. This indicates that the patch-level redundancy exists in various vision transformer models and our method can well excavate the redundancy.

We further conduct experiments on a SOTA transformer model, LV-ViT [20], and show the results in Table 2. The results show that our patch pruning method also work well on LV-ViT, *e.g.*, the dynamic patch slimming reduces the FLOPs of LV-ViT-M from 16.0G to 8.3G, still achieving 83.7% top-1 accuracy. Its performance is also superior to other SOTA models such as Swin transformer [26].

5.2. Ablation Study

We conduct extensive ablation studies on ImageNet to verify the effectiveness of each component in our method. The DeiT-S model on the ImageNet dataset is used as the base model.

The effect of global tolerant error ϵ. The tolerant error ϵ affects the balance between computational cost and accuracy of the pruned model, which is empirically investigated in Figure 4. Increasing ϵ implies larger reconstructed error between features of the pruned DeiT and original DeiT, while more patches can be pruned to achieve higher acceleration rate. When the reduction of FLOPs is less than 45%, there is almost no accuracy loss (less than 0.4%), which is because that a large number of patches are redundant.

Learned patch pruning *vs*. uniform pruning. In our method, the number of patches required in a specific layers is determined automatically via the global tolerant value ϵ. The architecture of the pruned DeiT model is shown in Figure 5. We can see that a pyramid-like architecture is obtained, where most of the patches in deep layers are pruned while more patches are preserved in shallow layers. To val-

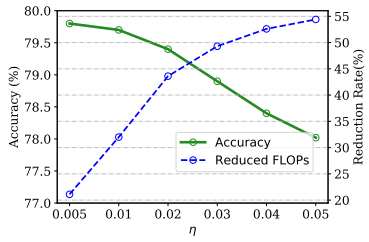


Figure 4. The ImageNet accuracy and FLOPs of the pruned DeiT-S w.r.t. tolerant error ϵ.

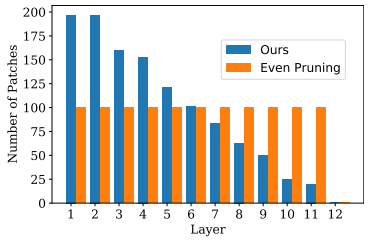


Figure 5. The architecture of pruned DeiT-S on ImageNet.

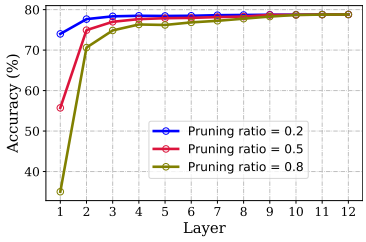
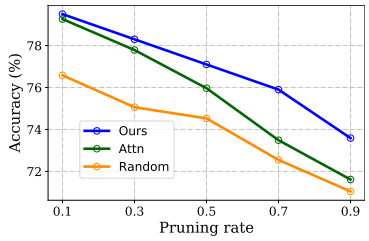
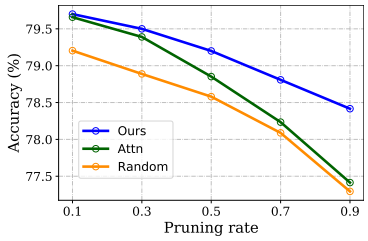


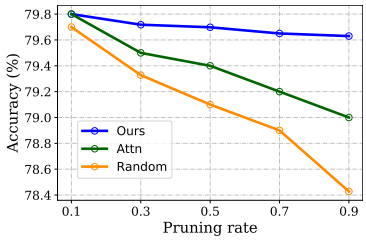
Figure 6. Pruning different layers of the DeiT-S model on ImageNet.



(a) The 3-th layer.



(b) The 7-th layer.



(c) The 11-th layer.

Figure 7. Accuracy w.r.t. the pruning rate of patches in a single layer.

idate the superiority of the learned pyramid architecture, we also implement a baseline that uniformly prunes all the layers with the similar pruning rate. We compare the results of the proposed patch slimming method and uniform pruning in Table 3. The accuracy of uniform pruning is only 77.2%, which incurs a large accuracy drop (-2.6%).

To better understand the behavior of patch pruning in the vision transformer, we prune patches in a single layer to see how the test accuracy change. The experiments are conducted with DeiT-S model on ImageNet.

Redundancy w.r.t. depth. We test the patch redundancy of different layers to verify the motivation of top-down patch slimming procedure. We prune a single layer and keep the same pruning ratio for different layers. Figure 6 shows the accuracy of the pruned model after pruning patches of a certain layer, and each line denotes pruning patches with a given pruning rate. In deeper layers, more patches can be safely removed without large impact on the final performance. However, removing a patch in lower layers usually incurs obvious accuracy drop. The patch redundancy is extremely different across layers and deeper layers have more redundancy, which can be attributed to that the attention mechanism aggregates features from different patches and the deeper patches have been fully communicated with each other. This phenomenon is different from the channel pruning in CNNs, where lower layers are observed to have more channel-level redundancy (Figure 4 in [16]).

Effectiveness of impact estimation. We define the scores s_l in Eq. 6 to approximate significance of a patch by propagating the reconstruction error of effective patches in output layer. To validate its effectiveness, we compare it with two baseline scores: 'Random' denotes removing patches in the layer randomly, and 'Attn' approximates the importance of a patch only with the norm of its attention map in the current layer. We compare the three scores by utilizing them to prune patches in different layer. The results are presented in Figure 7, where y-axis is the test accuracy of the pruned models (without fine-tuning). From the results, our impact estimation manner suffers less accuracy loss than the others with the same pruning rate (*e.g.*, 50%). It implies that our method can effectively identify patches that really make contributions to the final prediction.

6. Conclusion

We propose to accelerate vision transformers by reducing the number of patches required to calculate. Considering that the attention mechanism aggregates different patches layer-by-layer, a top-down framework is developed to excavate the redundant patches. The importance of each patch is also approximated according to its impact on the effective output features. After pruning, a compact vision transformer with a pyramid-like architecture is obtained. Extensive experiments on benchmark datasets validate that the proposed method can effectively reduce the computational cost. In the future, we plan to combine the patch slimming methods with more compression technologies (*e.g.*, weight pruning, model quantization) to explore extremely efficient vision transformers.

Acknowledgment. This work is supported by National Natural Science Foundation of China under Grant No.61876007, Australian Research Council under Project DP210101859 and the University of Sydney SOAR Prize.

References

[1] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020. 2

[2] Hanting Chen, Yunhe Wang, Tianyu Guo, Chang Xu, Yiping Deng, Zhenhua Liu, Siwei Ma, Chunjing Xu, Chao Xu, and Wen Gao. Pre-trained image processing transformer. *arXiv preprint arXiv:2012.00364*, 2020. 1

[3] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830*, 2016. 1

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 6

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 2

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1, 7

[7] Paul Dupuis and Hitoshi Ishii. On lipschitz continuity of the solution mapping to the skorokhod problem, with applications. *Stochastics: An International Journal of Probability and Stochastic Processes*, 35(1):31–62, 1991. 5

[8] Ken-ichi Funahashi and Yuichi Nakamura. Approximation of dynamical systems by continuous time recurrent neural networks. *Neural networks*, 6(6):801–806, 1993. 5

[9] Xitong Gao, Yiren Zhao, Lukasz Dudziak, Robert Mullins, and Cheng zhong Xu. Dynamic channel pruning: Feature boosting and suppression. In *International Conference on Learning Representations*, 2019. 6

[10] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. Power-bert: Accelerating bert inference via progressive word-vector elimination. In *International Conference on Machine Learning*, pages 3690–3699. PMLR, 2020. 6, 7

[11] Kai Han, Jianyuan Guo, Yehui Tang, and Yunhe Wang. Pyramidtnt: Improved transformer-in-transformer baselines with pyramid architecture. *arXiv preprint arXiv:2201.00978*, 2022. 1

[12] Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. Transformer in transformer. *arXiv preprint arXiv:2103.00112*, 2021. 7

[13] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015. 1

[14] Song Han, Jeff Pool, John Tran, and William J Dally. Learning both weights and connections for efficient neural networks. *arXiv preprint arXiv:1506.02626*, 2015. 1

[15] Yang He, Ping Liu, Ziwei Wang, Zhilan Hu, and Yi Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4340–4349, 2019. 2

[16] Yihui He, Xiangyu Zhang, and Jian Sun. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1389–1397, 2017. 1, 8

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 1

[18] Weizhe Hua, Yuan Zhou, Christopher De Sa, Zhiru Zhang, and G Edward Suh. Channel gating neural networks. *arXiv preprint arXiv:1805.12549*, 2018. 6

[19] Huawei. Mindspore. https://www.mindspore.cn/, 2020. 7

[20] Zihang Jiang, Qibin Hou, Li Yuan, Zhou Daquan, Yujun Shi, Xiaojie Jin, Anran Wang, and Jiashi Feng. All tokens matter: Token labeling for training better vision transformers. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021. 7

[21] Tae Hyun Kim, Mehdi SM Sajjadi, Michael Hirsch, and Bernhard Scholkopf. Spatio-temporal transformer network for video restoration. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 106–122, 2018. 1

[22] Xu Lan, Xiatian Zhu, and Shaogang Gong. Knowledge distillation by on-the-fly native ensemble. *arXiv preprint arXiv:1806.04606*, 2018. 1

[23] Vadim Lebedev and Victor Lempitsky. Fast convnets using group-wise brain damage. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2554–2564, 2016. 2

[24] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. *arXiv preprint arXiv:1608.08710*, 2016. 1

[25] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2736–2744, 2017. 1, 2, 3, 4

[26] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. *arXiv preprint arXiv:2103.14030*, 2021. 7

[27] Zhenhua Liu, Yunhe Wang, Kai Han, Wei Zhang, Siwei Ma, and Wen Gao. Post-training quantization for vision transformer. *Advances in Neural Information Processing Systems*, 34, 2021. 1

[28] Zhenhua Liu, Jizheng Xu, Xiulian Peng, and Ruiqin Xiong. Frequency-domain dynamic pruning for convolutional neural networks. *Advances in neural information processing systems*, 31, 2018. 1

[29] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, volume 32, 2019. 2

[30] Zizheng Pan, Bohan Zhuang, Jing Liu, Haoyu He, and Jianfei Cai. Scalable vision transformers with hierarchical pooling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 377–386, 2021. 6, 7

[31] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017. 7

[32] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European conference on computer vision*, pages 525–542. Springer, 2016. 1

[33] Xiu Su, Shan You, Tao Huang, Fei Wang, Chen Qian, Changshui Zhang, and Chang Xu. Locally free weight sharing for network width search. In *ICLR*. OpenReview.net, 2021. 2

[34] Xiu Su, Shan You, Fei Wang, Chen Qian, Changshui Zhang, and Chang Xu. Bcnet: Searching for network width with bilaterally coupled network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2175–2184, 2021. 2

[35] Yehui Tang, Kai Han, Chang Xu, An Xiao, Yiping Deng, Chao Xu, and Yunhe Wang. Augmented shortcuts for vision transformers. *Advances in Neural Information Processing Systems*, 34, 2021. 1

[36] Yehui Tang, Yunhe Wang, Yixing Xu, Yiping Deng, Chao Xu, Dacheng Tao, and Chang Xu. Manifold regularized dynamic network pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5018–5028, 2021. 6

[37] Yehui Tang, Yunhe Wang, Yixing Xu, Dacheng Tao, Chunjing Xu, Chao Xu, and Chang Xu. Scop: Scientific control for reliable neural network pruning. *Advances in Neural Information Processing Systems*, 33:10936–10947, 2020. 2, 3, 6, 7

[38] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. *arXiv preprint arXiv:2012.12877*, 2020. 1, 7

[39] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *arXiv preprint arXiv:1905.09418*, 2019. 2

[40] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. *arXiv preprint arXiv:2102.12122*, 2021. 7

[41] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. Learning structured sparsity in deep neural networks. *arXiv preprint arXiv:1608.03665*, 2016. 2

[42] Jianbo Ye, Xin Lu, Zhe Lin, and James Z Wang. Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers. *arXiv preprint arXiv:1802.00124*, 2018. 1

[43] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. *arXiv preprint arXiv:2101.11986*, 2021. 1, 7

[44] Mingjian Zhu, Kai Han, Yehui Tang, and Yunhe Wang. Visual transformer pruning. *arXiv preprint arXiv:2104.08500*, 2021. 2, 6, 7